

Modelo de documentação

Introdução

Descrição Geral do Projeto: A **ShopMind** é um sistema de gerenciamento de loja. A aplicação permite gerenciar produtos, clientes e pedidos, e conta com uma assistente virtual com Inteligência Artificial, que cria pedidos, verifica se existe endereço do cliente cadastrado, se existir ela usa o mesmo, caso não exista ela cria um novo para o cliente.

Objetivos: o Projeto visa alcançar uma certa rapidez, de forma que o cliente faça um pedido e seja atendido mais rápido pela atendente virtual

Escopo: O sistema ShopMind abrange o gerenciamento de produtos, clientes, endereços e pedidos, além de uma assistente virtual com IA capaz de criar pedidos e gerenciar endereços de forma automática durante o atendimento. A aplicação é disponibilizada via API REST e voltada para uso acadêmico em ambiente local.

Estão fora do escopo desta versão: integrações com meios de pagamento, rastreamento de entregas, aplicativo mobile e autenticação de usuários.

Visão Geral do Sistema

Arquitetura do Sistema: O sistema segue com arquitetura em camadas no backend e react no Frontend

Funcionalidades:

Gestão de Produtos

- Cadastrar novo produto (nome, descrição, preço, estoque)
- Listar todos os produtos
- Buscar produto por ID
- Editar informações de um produto
- Remover produto

Gestão de Clientes

- Cadastrar novo cliente (nome, e-mail, telefone)
- Listar todos os clientes
- Buscar cliente por ID

- Editar dados de um cliente
- Remover cliente

Gestão de Endereços

- Cadastrar endereço vinculado a um cliente
- Consultar endereço existente de um cliente
- Atualizar endereço de um cliente

Gestão de Pedidos

- Criar pedido manualmente
- Listar todos os pedidos
- Buscar pedido por ID
- Atualizar status de um pedido
- Cancelar pedido

Assistente Virtual com IA

- Receber mensagem do cliente via chat
- Interpretar a intenção do cliente (qual produto quer pedir, quantidade)
- Verificar se o cliente já possui endereço cadastrado
- Reutilizar o endereço existente ou solicitar um novo e cadastrá-lo automaticamente
- Criar o pedido automaticamente com base na conversa
- Responder ao cliente confirmando o pedido realizado

Configuração do Ambiente

Requisitos de Software e Hardware:

- .NET 8 SDK
- SQL Server Express
- Node.js 18+
- Visual Studio 2022 ou VS Code
- RAM: 4GB | Disco: 3GB | Windows 10+

Instruções de Instalação:

git clone <url-do-repositorio>

dotnet restore

No FrontEnd, npm install

Configuração do Ambiente de Desenvolvimento: Preencha o appsettings.json com sua connection string, chave Groq e chave JWT, depois rode:

dotnet ef database update

dotnet run --project ProjetoLoja.API

npm start, no frontend

Dependências:

Dependências Back-end

- EF Core + SqlServer - banco de dados
- Swashbuckle - Swagger
- JwtBearer + Tokens.Jwt - autenticação
- BCrypt.Net-Next - hash de senhas

Dependências Front-end

- axios - requisições HTTP
- react-router-dom - navegação
- bootstrap + react-bootstrap - estilização
- react-icons - ícones

.

Desenvolvimento

Estrutura do Projeto:

O projeto foi dividido em 5 camadas, cada uma com uma responsabilidade diferente:

ProjetoLoja.API - é a entrada do sistema, onde ficam os controllers e os modelos de requisição e resposta.

ProjetoLoja.Dominio - contém as entidades do sistema, como Cliente, Produto, Pedido, Endereços, itens Pedidos, tipo Produto e Tipos Usuários.

ProjetoLoja.Repositorio - responsável pelo acesso ao banco de dados usando Entity Framework Core.

ProjetoLoja.Aplicacao - onde ficam as regras de negócio e a lógica da aplicação.

ProjetoLoja.Servicos - integra serviços externos, como a IA da Groq.

Descrição das Camadas:

ProjetoLoja.API - Apresentação É a camada de entrada do sistema. Aqui ficam os controllers que recebem as requisições HTTP, os modelos de dados (DTOs) e a configuração geral da aplicação no Program.cs, incluindo a autenticação JWT e o Swagger.

ProjetoLoja.Dominio - Domínio Contém as entidades do sistema, como Cliente, Produto, Pedido, Endereco e ItensPedido. Essa camada não depende de nenhuma outra, ela representa as regras e estruturas centrais do negócio.

ProjetoLoja.Repositorio - Repositório Responsável pelo acesso ao banco de dados. Usa o Entity Framework Core para fazer as operações de leitura e escrita no SQL Server através do DbContext.

ProjetoLoja.Aplicacao - Aplicação Onde ficam os casos de uso e a lógica de negócio, como a criação de pedidos e a validação de clientes. Também é aqui que o BCrypt é usado para proteger as senhas.

ProjetoLoja.Servicos - Serviços Camada responsável pelas integrações externas. Aqui fica a comunicação com a API da Groq para o funcionamento da assistente virtual.

API

Documentação da API:

Cliente: GET/api/cliente Lista todos os clientes
GET/api/cliente/{id} Busca cliente por ID
POST/api/cliente Cria novo cliente
PUT/api/cliente/{id} Atualiza cliente
DELETE/api/cliente/{id} Remove cliente

Produto: GET/api/produto Lista todos os produtos
GET/api/produto/{id} Busca produto por ID
POST/api/produto Cria novo produto
PUT/api/produto/{id} Atualiza produto
DELETE/api/produto/{id} Remove produto

Pedido: GET/api/pedido Lista todos os pedidos
GET/api/pedido/{id} Busca pedido por ID
POST/api/pedido Cria novo pedido
PUT/api/pedido/{id} Atualiza pedido
DELETE/api/pedido/{id} Cancela pedido

Endereço: GET/api/endereco/{clienteId}Busca endereço do cliente
POST/api/enderecoCadastra endereço
PUT/api/endereco/{id}Atualiza endereço
DELETE/api/endereco/{id}Remove endereço

ItensPedido: GET/api/itenspedido/{pedidoId}Lista itens do pedido
POST/api/itenspedidoAdiciona item ao pedido
PUT/api/itenspedido/{id}Atualiza item
DELETE/api/itenspedido/{id}Remove item

TipoProduto: GET/api/tipoprodutoLista tipos de produto
POST/api/tipoprodutoCria tipo
PUT/api/tipoproduto/{id}Atualiza tipo
DELETE/api/tipoproduto/{id}Remove tipo

Login: POST/api/loginAutentica e retorna JWT

IA: POST/api/ia/chatEnvia mensagem para a assistente virtual

Interface do Usuário

Descrição das Funcionalidades da Interface:

Login Tela inicial do sistema onde o usuário informa seu e-mail e senha para acessar a aplicação. Após autenticação, o token JWT é armazenado e utilizado nas demais requisições.

Dashboard Página inicial após o login, com uma visão geral do sistema para navegação entre as funcionalidades.

Clientes Lista todos os clientes cadastrados no sistema, com opções para editar ou acessar detalhes de cada um.

Novo Cliente Formulário para cadastrar um novo cliente, informando nome, e-mail e senha.

Editar Cliente Permite atualizar as informações de um cliente já cadastrado.

Produtos Exibe todos os produtos disponíveis na loja, com nome, preço e estoque.

Novo Produto Formulário para cadastrar um novo produto no sistema.

Editar Produto Permite alterar as informações de um produto existente.

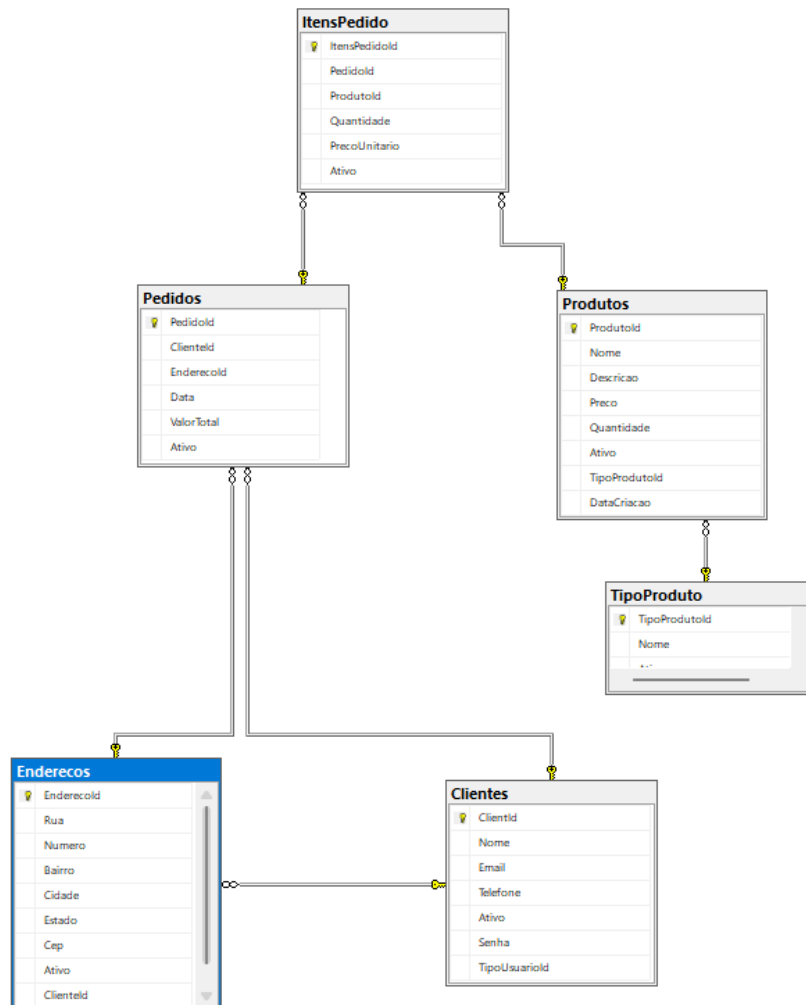
Pedidos Lista todos os pedidos realizados, com informações de status e cliente.

Novo Pedido Formulário para criação manual de um pedido, selecionando cliente, endereço e produtos.

Assistente IA Tela de chat onde o cliente conversa com a assistente virtual. A IA interpreta a mensagem, verifica se o cliente já tem endereço cadastrado e cria o pedido automaticamente.

Banco de Dados

Diagrama de Entidade-Relacionamento (ERD):



Considerações Finais

Lições Aprendidas: Durante o desenvolvimento do projeto aprendi bastante sobre autenticação JWT, entendendo como proteger endpoints e gerenciar tokens de forma segura. Também evolui muito em boas práticas de organização de código, como trabalhar com arquitetura em camadas para evitar que o projeto vire uma bagunça conforme cresce.

Uma prática que adotei foi a de desenvolvimento incremental: na primeira versão entrego apenas o que foi pedido, e nas versões seguintes vou aprimorando conforme surgem novas ideias. Isso ajudou a manter o foco e a qualidade do código.

Também aprendi a consumir APIs externas e a "treinar" uma IA através de prompts, definindo o comportamento da assistente virtual para que ela fizesse exatamente o que o sistema precisava

Melhores Práticas:

Arquitetura em camadas, separando responsabilidades entre API, Aplicação, Domínio, Repositório e Serviços

Uso de DTOs (Models/Requisicao e Models/Resposta) para não expor as entidades do banco diretamente na API

Chaves sensíveis como a API da Groq e a chave JWT armazenadas no appsettings.json, fora do código-fonte

Senhas dos usuários protegidas com hash BCrypt

Documentação automática dos endpoints via Swagger

Próximos Passos: Integração com meios de pagamento, como Pix e cartão de crédito

- Desenvolvimento de aplicativo mobile para acesso pelo celular
- Geração de relatórios de vendas e controle de estoque
- Expandir as capacidades da assistente virtual, dando a ela acesso a mais funcionalidades do sistema, como cadastrar e atualizar produtos automaticamente quando solicitado por um administrador logado
- Criar uma área exclusiva para clientes, onde possam acompanhar seus pedidos e histórico de compras
- Implementar níveis de acesso diferenciados, separando as permissões de administrador e cliente dentro do sistema.

Anexos

Referências e Recursos Adicionais: O desenvolvimento do projeto contou com o auxílio da inteligência artificial Claude para tirar dúvidas, organização do código e apoio nas decisões técnicas ao longo do projeto.

Links Úteis:

Documentação oficial do .NET 8: <https://learn.microsoft.com/dotnet>

Documentação do Entity Framework Core: <https://learn.microsoft.com/ef/core>

Documentação do Swagger: <https://swagger.io/docs>

API Groq: <https://console.groq.com/docs>

Claude (IA utilizada): <https://claude.ai>

Repositório do Projeto BackEnd: <https://github.com/duh1721/Projeto-Final-Loja>

Repositório do Projeto FrontEnd: <https://github.com/duh1721/ProjetoFinalLoja.App>

Créditos e Agradecimentos: Projeto desenvolvido individualmente. Agradeço ao monitor da itera pela orientação durante o desenvolvimento e à ferramenta Claude pelo suporte técnico ao longo do projeto.